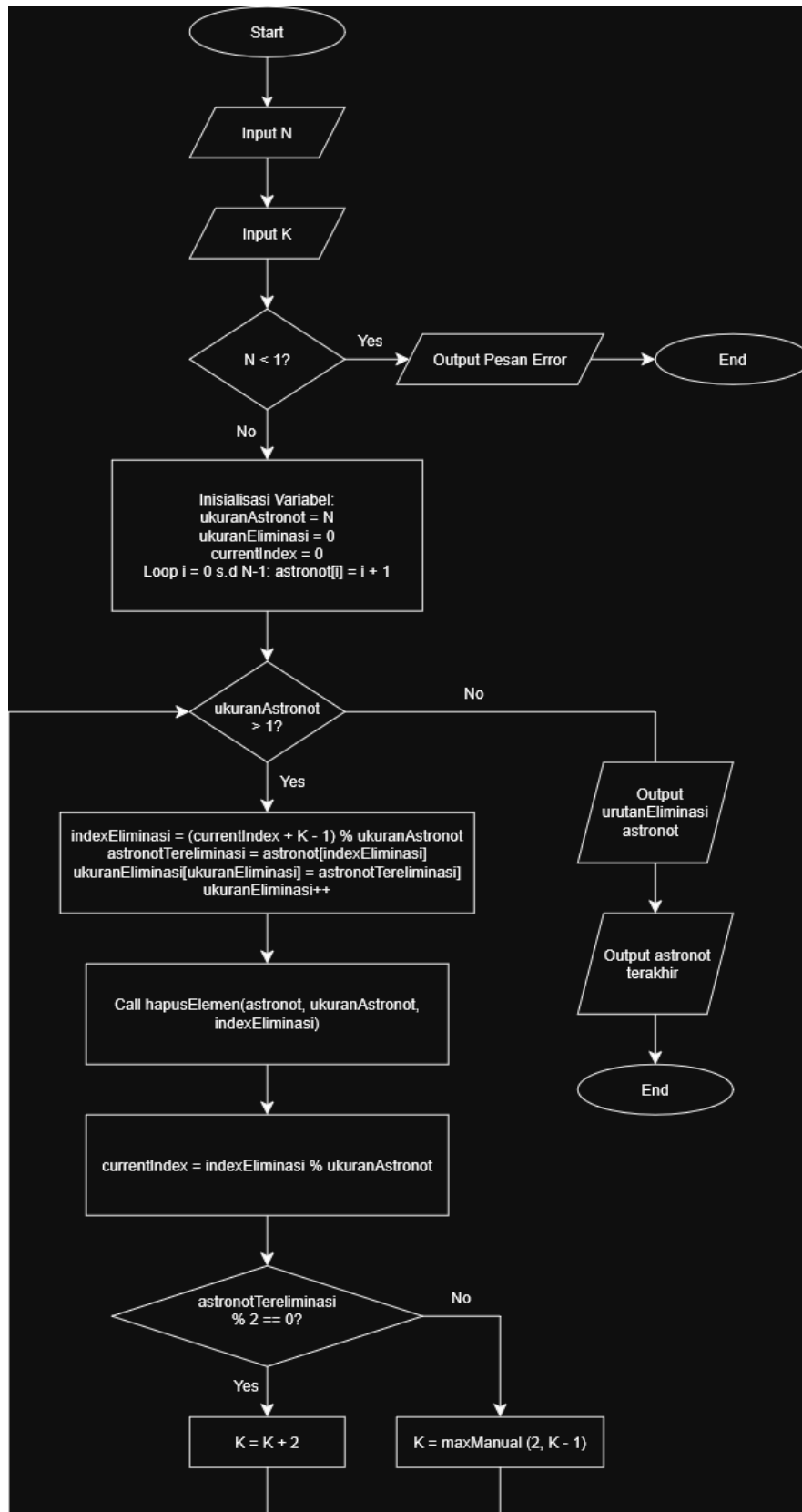


PROBLEM 1: THE LAST ASTRONAUT

Flowchart:



Penjelasan Logika:

Program diawali dengan membaca dua input, yaitu jumlah astronot (N) dan K. Langkah awal merupakan validasi untuk memastikan bahwa jumlah astronot minimal bernilai 1. Jika nilai "N" kurang dari 1, sistem akan menampilkan pesan error dan mengakhiri program. Jika nilai "N" valid, program melanjutkan ke tahap inisialisasi variabel. Pada tahap ini, daftar astronot diberi nomor urut 1 hingga N, ukuran daftar astronot diatur oleh banyak N, ukuran daftar eliminasi dimulai dari 0, serta penunjuk posisi (currentIndex) diset awal pada indeks 0.

Proses utama dijalankan menggunakan struktur perulangan yang akan terus berulang selama jumlah astronot yang tersisa masih lebih dari satu. Di dalam setiap iterasi perulangan, program menentukan posisi astronot yang akan tereliminasi berdasarkan rumus $(currentIndex + K - 1) \bmod ukuranAstronot$. Astronaut pada posisi tersebut kemudian dicatat nomornya ke dalam daftar "urutanEliminasi", dan jumlah data eliminasi bertambah satu. Selanjutnya, program memanggil fungsi "hapusElemen" untuk mengeluarkan astronot tersebut dari lingkaran sekaligus menggeser posisi elemen array dan mengurangi jumlah total astronot yang tersisa.

Setelah proses penghapusan elemen selesai, posisi "currentIndex" diperbarui untuk menyesuaikan urutan giliran berikutnya. Sebagai aturan dinamika permainan, sistem memeriksa nomor ID dari astronot yang baru saja tereliminasi. Jika nomor ID astronot tersebut genap, nilai K akan bertambah sebanyak 2 unit ($K = K + 2$). Sebaliknya, jika nomor ID astronot tersebut ganjil, nilai K disesuaikan dengan nilai terbesar antara 2 dan $K - 1$ menggunakan fungsi "maxManual". Perulangan ini terus berulang dan memperbarui nilai K hingga tersisa tepat satu orang astronot.

Ketika jumlah astronot yang tersisa sudah bernilai satu (kondisi perulangan tidak lagi terpenuhi), alur program keluar dari lingkaran perulangan menuju tahap akhir. Program kemudian mencetak urutan lengkap nomor astronot yang telah tereliminasi secara berurutan, diikuti dengan menampilkan nomor ID dari astronot terakhir yang bertahan hidup di dalam array. Setelah seluruh data hasil simulasi ditampilkan ke layar, alur program secara resmi berakhir.

PROBLEM 2: ALIEN IN THE MIDDLE

Pseudocode:

```
FUNCTION apakahHuruf(c : CHARACTER) : BOOLEAN
    IF (c >= 'A' AND c <= 'Z') OR (c >= 'a' AND c <= 'z') THEN[cite: 3]
        RETURN TRUE
    ELSE[cite: 3]
        RETURN FALSE
    END IF[cite: 3]
END FUNCTION
```

```
FUNCTION apakahKapital(c : CHARACTER) : BOOLEAN
    IF (c >= 'A' AND c <= 'Z') THEN[cite: 3]
        RETURN TRUE
    ELSE[cite: 3]
        RETURN FALSE
    END IF[cite: 3]
END FUNCTION
```

```
FUNCTION ubahKeKapital(c : CHARACTER) : CHARACTER
    IF (c >= 'a' AND c <= 'z') THEN[cite: 3]
        RETURN c - ('a' - 'A')
    END IF[cite: 3]
    RETURN c
END FUNCTION
```

```
FUNCTION hitungPanjang(teks : ARRAY OF CHARACTER) : INTEGER
    DECLARE panjang : INTEGER[cite: 3]
    SET panjang = 0[cite: 3]

    WHILE teks[panjang] != '\0' DO[cite: 3]
        SET panjang = panjang + 1[cite: 3]
    END WHILE[cite: 3]

    RETURN panjang
END FUNCTION
```

```
PROCEDURE enkripsiSandiAstronot(teks : ARRAY OF CHARACTER, hasil : ARRAY  
OF CHARACTER)
```

```
    DECLARE panjang, i, nilaiPergeseran, posisiAsli, posisiBaru : INTEGER[cite: 3]
```

```
    DECLARE isUpper : BOOLEAN[cite: 3]
```

```
    DECLARE hurufAcuan, hurufSaatIni : CHARACTER[cite: 3]
```

```
    SET panjang = hitungPanjang(teks)[cite: 3]
```

```
    IF panjang == 0 THEN[cite: 3]
```

```
        SET hasil[0] = '\0'[cite: 3]
```

```
        RETURN
```

```
    END IF[cite: 3]
```

```
    SET hasil[0] = teks[0][cite: 3]
```

```
    FOR i = 1 TO panjang - 1 DO[cite: 3]
```

```
        IF apakahHuruf(teks[i]) THEN[cite: 3]
```

```
            SET isUpper = apakahKapital(teks[i])[cite: 3]
```

```
            SET hurufAcuan = ubahKeKapital(teks[i - 1])[cite: 3]
```

```
            SET nilaiPergeseran = hurufAcuan - 'A' + 1[cite: 3]
```

```
            SET hurufSaatIni = ubahKeKapital(teks[i])[cite: 3]
```

```
            SET posisiAsli = hurufSaatIni - 'A'[cite: 3]
```

```
            SET posisiBaru = (posisiAsli + nilaiPergeseran) MOD 26[cite: 3]
```

```
            IF isUpper THEN[cite: 3]
```

```
                SET hasil[i] = posisiBaru + 'A'[cite: 3]
```

```
            ELSE[cite: 3]
```

```
                SET hasil[i] = posisiBaru + 'a'[cite: 3]
```

```
            END IF[cite: 3]
```

```
        ELSE[cite: 3]
```

```
            SET hasil[i] = teks[i][cite: 3]
```

```
        END IF[cite: 3]
```

```
    END FOR[cite: 3]
```

```
    SET hasil[panjang] = '\0'[cite: 3]
```

```
END PROCEDURE
```

```

START[cite: 3]
    DECLARE inputTeks : ARRAY[100] OF CHARACTER[cite: 3]
    DECLARE hasilEnkripsi : ARRAY[100] OF CHARACTER[cite: 3]

    PRINT "Masukkan kata/pesan: "[cite: 3]
    INPUT inputTeks[cite: 3]

    CALL enkripsiSandiAstronot(inputTeks, hasilEnkripsi)

    PRINT "Hasil Enkripsi: "[cite: 3]
    PRINT hasilEnkripsi[cite: 3]
END[cite: 3]

```

Penjelasan Logika:

Program diawali dengan membaca masukan teks dari pengguna yang akan disandi. Selanjutnya, sistem menjalankan fungsi “hitungPanjang” untuk menentukan total jumlah karakter dari teks masukan tersebut. Tahap awal berupa validasi dilakukan untuk memeriksa panjang teks, jika panjang teks bernilai 0 (kosong), program akan mengisi karakter null terminator (`\0`) pada array hasil dan langsung mengakhiri fungsi. Apabila teks memiliki isi, karakter pertama dari teks asal (`teks[0]`) langsung disalin ke variabel hasil (`hasil[0]`) tanpa mengalami perubahan pergeseran nilai.

Proses utama enkripsi dijalankan menggunakan perulangan FOR yang memproses karakter mulai dari indeks ke-1 hingga karakter sebelum null terminator. Di setiap iterasi, program mengecek tipe karakter menggunakan fungsi “apakahHuruf”. Jika karakter saat ini berupa huruf, sistem menentukan status kapitalnya “apakahKapital” dan mengambil huruf asli dari indeks sebelumnya (`teks[i-1]`) untuk dijadikan sebagai acuan pergeseran. Nilai pergeseran dihitung dari urutan alfabet huruf acuan tersebut, lalu ditambahkan ke posisi alfabet asli dari karakter saat ini. Hasil penjumlahan ini diproses menggunakan operasi modulo 26 (`% 26`) untuk memastikan pergeseran tetap berputar dalam rentang alfabet A–Z. Karakter baru hasil pergeseran kemudian disimpan ke dalam array “`hasil[i]`”. Sebaliknya, jika karakter pada indeks saat ini bukan huruf (seperti spasi atau simbol), karakter tersebut disalin langsung ke array hasil tanpa mengalami perubahan.

Setelah seluruh karakter di dalam teks berhasil diproses melalui perulangan, program menambahkan penanda null terminator (`\0`) pada indeks “`hasil[panjang]`” untuk menandai batas akhir string hasil enkripsi. Alur program kemudian kembali ke fungsi utama (main) untuk menampilkan hasil enkripsi teks yang telah diproses ke layar pengguna, lalu seluruh proses enkripsi berakhir secara resmi.